

Lesson 16: Sequential and Parallel Workflows

Agentic Systems Design

Complex agent tasks often contain several smaller pieces of work.

Some pieces must happen in a particular order:

```

Authenticate user
↓
Retrieve order
↓
Check eligibility
↓
Issue approved refund
  
```

Other pieces can happen independently:

```

■■■ Search flights
Plan trip ■■■■■■ Search hotels
■■■ Read travel policy
  
```

These two execution patterns are called:

Sequential workflow:

- Tasks run in a required order.

Parallel workflow:

- Independent tasks run concurrently.

Most production agent systems use a combination of both.

The central principle is:

Run tasks sequentially when one task depends on another. Run tasks in parallel only when their inputs, permissions and side effects are sufficiently independent.

1. What Is a Workflow?

A workflow is an organized sequence or network of steps used to accomplish a goal.

A workflow defines:

- Which steps exist
- Which steps depend on others
- Which steps may run concurrently
- What data each step receives
- What outputs each step produces
- What happens when a step fails
- When human approval is required
- What counts as completion

Example:

Goal: Resolve a damaged-product complaint

Possible workflow:

- Identify customer
- ↓ Retrieve order
- ↓ Collect damage evidence
- ↓ Check policy and inventory
- ↓ Present resolution
- ↓ Obtain confirmation
- ↓ Create replacement
- ↓ Verify completion

2. Workflow versus Agent

A workflow determines the execution structure. An agent may make decisions within that structure.

Workflow: Defines the permitted process.

Agent: Interprets observations and chooses among permitted actions.

Example – Deterministic workflow:

1. Authenticate employee.
2. Retrieve payroll record.
3. Calculate expected payment.
4. Compare actual payment.
5. Escalate when discrepancy exceeds threshold.

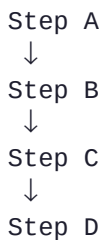
The model may help with:

- Interpreting the employee's complaint
- Extracting the affected pay period
- Explaining the discrepancy
- Deciding which additional evidence is useful

The workflow still controls mandatory stages.

3. Sequential Workflow

A sequential workflow executes steps one after another.



Step B normally begins only after Step A reaches an acceptable result.

Example:

- Retrieve appointment
 - ↓ Find replacement slots
 - ↓ Ask user to select
 - ↓ Request confirmation
 - ↓ Reschedule appointment

Each stage depends on information produced by the previous stage.

4. Why Sequential Execution Is Necessary

Sequential execution is required when:

- A later step needs an earlier output.
- An earlier step determines whether a later step is allowed.
- Operations must occur in a business-defined order.
- A write must be verified before another write.
- Human confirmation must precede execution.
- Transactional consistency matters.

Example: You cannot refund an order before identifying and verifying the order.

Another: You should not send an appointment confirmation before the appointment exists.

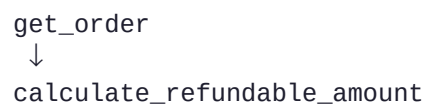
5. Data Dependency

A data dependency exists when one step requires the output of another.

Example:

```
Step 1: get_order(order_id)
Output: {
  "amount_paid_minor": 34999,
  "currency": "CAD"
}
Step 2: calculate_refundable_amount
```

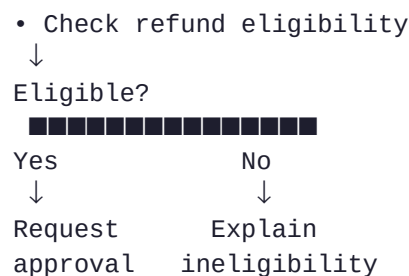
The second step needs the first step's result.
Therefore:



They cannot safely run in parallel.

6. Control Dependency

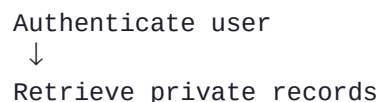
A control dependency exists when one step determines whether another step should run.



Even when a later step does not require the earlier step's raw data, it may require its decision.

7. Authorization Dependency

Some steps depend on authorization.



Or:

- Request manager approval
- ↓
- Issue refund

These are not merely data dependencies. They are **permission dependencies**. The application must ensure that the authorized event occurred before exposing or executing the next capability.

8. State Dependency

A workflow step may require a particular run state.

Example:

```
reschedule_appointment
Allowed only when:
{
  "run_status": "ready_to_execute",
  "appointment_verified": true,
  "slot_verified": true,
  "approval_status": "approved"
}
```

The workflow should reject execution from an earlier state such as `SEARCHING_AVAILABILITY`.

9. Sequential Pipeline

A simple sequential pipeline passes output from one step into the next.

```
Input
↓ Step A
↓ Output A
↓ Step B
↓ Output B
↓ Step C
↓ Final output
```

Example document-processing pipeline:

- Upload document
- Validate file type
- Extract text
- Identify relevant sections
- Summarize
- Validate output

10. Sequential Agent Pattern

An agent may control some decisions in a sequential process.

```
Goal
↓ Model selects next approved action
↓ Tool executes
↓ State updates
↓ Model receives new observation
↓ Next action
```

This remains sequential because every new model decision depends on the preceding observation.

The execution order is:

```
Model call 1
↓ Tool result 1
↓ Model call 2
↓ Tool result 2
```

The second model call cannot correctly occur before the first tool result exists.

11. Sequential Workflow Advantages

Sequential workflows provide:

- Clear ordering
- Easier reasoning
- Simpler state management
- Predictable business-rule enforcement
- Easier debugging
- Easier auditing
- Natural approval gates
- Reduced race conditions

They are appropriate for:

- Financial transactions
- Account modifications
- Approval processes
- Ordered data transformations
- State-machine transitions
- Tasks with strong dependencies

12. Sequential Workflow Disadvantages

Sequential execution may cause:

- Higher total latency
- Idle time while external services respond
- Underuse of independent resources
- Slow research or aggregation
- One slow step delaying everything

Example:

Search source A: 4 seconds
 ↓ Search source B: 5 seconds
 ↓ Search source C: 3 seconds
 Sequential total: ≈ 12 seconds

If the searches are independent, parallel execution may reduce the waiting time.

13. What Is Parallel Execution?

Parallel execution means that two or more independent tasks are active during overlapping periods.

■■■ Task A ■■■
 Input ■■■■ Task B ■■■■ Aggregate
 ■■■ Task C ■■■

This pattern is often called:

- Fan-out $\rightarrow$ Parallel work $\rightarrow$ Fan-in

14. Fan-Out

Fan-out creates several branches from one parent step.

```
graph TD; A[Research question] --> B[ ]; B --> C[Legal]; B --> D[Financial]; B --> E[Technical]
```

Each branch receives a bounded subtask:

- Legal branch: Identify relevant regulatory risks.
- Financial branch: Analyse financial impact.
- Technical branch: Assess implementation feasibility.

15. Fan-In

Fan-in combines results from several branches.

Legal result ██████
Financial result ██████ Aggregator
Technical result ████

The aggregator may:

- Merge results
- Compare conclusions
- Resolve conflicts
- Rank options
- Identify missing work
- Produce the final report

Fan-in should not simply concatenate every output without validation.

16. When Tasks Are Parallelizable

Two tasks are good candidates for parallel execution when:

- Neither requires the other's output.
- Neither changes state required by the other.
- They do not compete unsafely for the same resource.
- Their permissions can be independently bounded.
- Their outputs can be combined deterministically or safely interpreted.
- Failure in one branch does not corrupt the others.

Example: Search flights, Search hotels, Read travel policy — these can usually begin from the same travel requirements.

17. When Tasks Should Not Run in Parallel

Do not parallelize when:

- One task determines the input of another.
- One task authorizes another.
- Two tasks modify the same resource.
- The correct ordering is legally or operationally required.
- One task may invalidate another's assumptions.
- Duplicate or conflicting side effects are possible.

Unsafe: Check whether refund is permitted and Issue refund in parallel. The refund might execute before eligibility is known.

18. Independence Test

Before parallelizing two tasks, ask:

- Can Task B execute correctly using only the information available before Task A finishes?

If no, they are dependent.

Also ask:

- Could Task A's result cause Task B to be skipped, changed or prohibited?

If yes, sequential execution is safer.

19. Read–Read Parallelism

Two read-only operations are often safe to run in parallel. Example:

```
get_order_tracking
get_refund_policy
get_product_details
```

These retrieve different information and do not intentionally change the environment. However, even read operations require:

- Authorization
- Data minimization
- Rate limiting
- Result freshness
- Error handling

Read-only does not automatically mean unrestricted.

20. Read–Write Parallelism

A read and a write may conflict. Example: Read current appointment and Cancel appointment running simultaneously.

Possible consequences:

- Stale results
- Incorrect final communication
- Failed verification
- Race conditions

A common safer order is:

```
Read and verify
↓ Execute write
↓ Read again to verify
```

21. Write–Write Parallelism

Parallel writes are dangerous when they affect the same resource. Example: Update shipping address and Cancel order running concurrently.

Possible outcomes:

- Address is changed after cancellation.
- Cancellation uses an outdated version.
- One write overwrites the other.
- State becomes inconsistent.

Write–write parallelism should normally require:

- Independent resources
- Transactions
- Locks
- Version checks
- Idempotency
- Explicit conflict policies

22. Safe Independent Writes

Some writes may safely run in parallel when they affect separate resources. Example:

- Create internal audit note
- Send approved notification

Even here, consider whether one depends on the other. If the audit note must prove that the notification was sent, they should not run independently. The correct design depends on the required semantics.

23. Sequential Consistency

A sequential workflow creates a clear ordering: A happened before B, B happened before C. This simplifies reasoning about state.

```
Approval received
↓ Refund submitted
↓ Refund verified
```

The event ordering establishes that the refund did not precede approval.

29. Barrier

A barrier is a synchronization point where execution waits for specified branches.

Task A ████

Task B ██████ Barrier → Continue

Task C ████

The barrier may require:

- All branches
- Any one branch
- A minimum number
- All required branches while optional branches may time out

30. Wait-for-All Barrier

A wait-for-all barrier continues only when every required branch completes.

Legal result

Financial result

Technical result

↓ All available

↓ Generate combined report

Use this when every perspective is mandatory.
Problem: One slow branch delays the entire workflow.
A timeout policy is still necessary.

31. Wait-for-Any Pattern

A wait-for-any pattern continues when the first acceptable result arrives.

Search primary inventory ████

Search backup inventory ██████ → First valid result

Search regional cache ██████████

After one result is accepted, remaining tasks may be cancelled. This pattern is useful for:

- Redundant services
- Low-latency retrieval
- Multiple equivalent providers

It is dangerous when sources are not equally authoritative.

32. Quorum Pattern

A quorum requires a minimum number of agreeing or successful branches.

Example 1: Three independent document extractors — at least two agree — Accept extracted value.

Example 2: Five regional status checks — at least three report outage — Declare widespread incident.

Quorum logic should be deterministic and designed for the specific domain.

33. Required and Optional Branches

Not every branch must block completion. Example research workflow:

Required: Official company filing, Government data

Optional: Industry commentary, Expert interviews

If an optional branch fails, the report may still complete with a limitation.

```
{
  "required_branches_complete": true,
  "optional_branches_failed": [
    "expert_interview_search"
  ]
}
```

34. Map Pattern

The map pattern applies the same operation to many independent items.

Documents: D1, D2, D3, D4

↓ Summarize each independently

S1, S2, S3, S4

```
summaries = parallel_map(summarize_document, documents)
```

Other examples:

- Analyse each transaction
- Translate each paragraph
- Test each software module
- Compare each supplier
- Extract data from each invoice

35. Reduce Pattern

The reduce pattern combines mapped outputs.

```
Document summaries
↓ Compare themes
↓ Create consolidated report
```

The reducer may calculate:

- Totals
- Rankings
- Consensus
- Common themes
- Conflicts
- Final recommendations

The full structure is: Map → Parallel processing → Reduce

36. Map–Reduce Agent Pattern

Example research agent:

- Research question
 - ↓ Generate subquestions
 - ↓ Map: Research each subquestion independently
 - ↓ Reduce: Combine evidence into one supported conclusion

Potential branches: Market size, Competitors, Customer behaviour, Regulation, Technology.

The reducer should receive structured outputs rather than unrestricted prose where possible.

37. Scatter–Gather Pattern

Scatter–gather sends the same or related request to several workers and gathers their responses.

- Question
 - Agent A analysis
 - Agent B analysis
 - Agent C analysis
 - ↓ Comparison or synthesis

Uses include: Independent expert opinions, Diverse search strategies, Redundant verification, Candidate solution generation.

This can improve coverage but increases cost, coordination complexity, contradictions, and duplicate work.

38. Parallel Specialists

A manager agent may delegate to specialists.

```
Manager
■■■ Financial specialist
■■■ Legal specialist
■■■ Technical specialist
```

Each specialist should receive:

- A bounded goal
- Relevant context only
- A restricted toolset
- A time and cost budget
- An output schema
- Clear completion criteria

Do not tell each specialist merely: "Analyse everything."

39. Specialist Output Contract

A structured child output:

```
{
  "specialist": "legal",
  "status": "completed",
  "findings": [{
    "claim": "Additional consent may be required.",
    "evidence_refs": ["DOC-14"]
  }],
  "risks": [{
    "severity": "high",
    "description": "Potential cross-border data re
  }],
  "uncertainties": [],
  "recommended_actions": [
    "Obtain jurisdiction-specific legal review."
  ]
}
```

A consistent schema simplifies aggregation.

40. Aggregation

Aggregation combines branch results into a useful outcome.

Possible aggregation methods:

- Concatenation
- Deduplication
- Ranking
- Weighted scoring
- Voting
- Rule-based merging
- Model-based synthesis
- Deterministic calculation

The correct method depends on the task.

41. Deterministic Aggregation

Use deterministic aggregation when the merge rule is exact.

Examples: Sum transaction amounts, Find minimum price, Combine unique identifiers, Sort appointments by time, Calculate average latency.

```
total_minor = sum(
    item["amount_minor"] for item in branch_results
)
```

Do not use a model for exact arithmetic or predictable set operations.

42. Probabilistic Aggregation

Use a model when combining results requires semantic judgment.

Examples: Synthesize legal/financial/technical risks, Compare conflicting research, Remove semantically duplicate findings, Explain trade-offs, Produce a coherent report.

The aggregator model should still operate within deterministic boundaries:

- Only approved branch outputs
- Citation preservation
- Output schema
- Maximum length
- Conflict reporting
- No invented evidence

43. Hybrid Aggregation

A strong aggregation process may be:

- Validate branch schemas
 - ↓ Remove failed or unauthorized outputs
 - ↓ Deterministically calculate metrics
 - ↓ Detect exact contradictions
 - ↓ Model synthesizes remaining evidence
 - ↓ Validate citation coverage

The model performs semantic synthesis. Code handles exact validation and calculation.

44. Conflicting Branch Results

Parallel branches may disagree. The aggregator should not silently select the most convenient answer.

It should:

- Identify the disagreement.
- Compare sources and assumptions.
- Determine whether one source is more authoritative.
- Request further investigation when needed.
- Preserve uncertainty in the final result.
- Escalate consequential unresolved conflicts.

45. Conflict Metadata

Branch results should record: Source, Time, Assumptions, Confidence or uncertainty, Policy version, Data version, Scope.

```
{
  "claim": "The proposed process is compliant.",
  "source": "internal_policy_v4",
  "jurisdiction": "Ontario",
  "assumptions": [
    "No customer data leaves Canada."
  ]
}
```

A conflicting branch may use a different assumption. The disagreement may disappear once assumptions are aligned.

46. Partial Failure

In a parallel workflow, one branch may fail while others succeed.

The workflow must decide:

- Retry the failed branch
- Continue without it
- Use a fallback
- Return partial results
- Escalate
- Stop the whole workflow

The correct decision depends on whether the branch is mandatory.

47. Fail-Fast Strategy

A fail-fast workflow stops remaining work when a critical branch fails.

```
Authentication fails
↓ Cancel private-data branches
↓ Stop
```

Another: Travel-policy retrieval fails → Do not continue to booking.

Fail-fast saves resources and prevents unsafe continuation.

48. Best-Effort Strategy

A best-effort workflow preserves successful branches and continues when possible.

Example: Research five markets. One data source is unavailable. The workflow may:

- Complete analysis for four markets
- Report the missing market
- Mark the result partially complete

Best-effort is appropriate when partial output is genuinely useful and safe.

49. All-or-Nothing Strategy

Some workflows require every critical stage to succeed. Example financial transfer:

- Debit source account
- Credit destination account
- Record transaction

These operations should behave transactionally. The workflow must not leave the source debited but the destination not credited. This requires database or payment-system transaction guarantees.

50. Compensation Pattern

When a completed action must be undone after a later failure, the workflow may run a compensating action.

Example travel booking: Flight booked successfully → Hotel booking fails → Cancel flight reservation.

This is part of a saga pattern. Compensation is not always a perfect rollback:

- Cancellation fees may apply.
- Availability may be lost.
- Notifications may already have been sent.
- Some actions are irreversible.

51. Saga Pattern

A saga manages a sequence of distributed writes using local transactions and compensating actions.

```
Reserve flight → Reserve hotel
→ Reserve car → Charge payment
```

```
Failure: Car reservation fails
Compensation (reverse):
Release hotel → Release flight
```

Every step should define:

- Forward action
- Completion evidence
- Compensation action
- Compensation eligibility
- Idempotency key
- Failure policy

52. Parallel Write Compensation

Parallel writes complicate compensation. Suppose:

```
Book flight ■■■■
Book hotel ■■■■■■ Both expected
Book car ■■■■■■
```

Flight: success Hotel: fails Car: success

The workflow must determine whether to: Cancel flight and car, Keep successful reservations, Ask the user, Retry hotel, or Escalate.

This policy should be defined before execution, not improvised after failure.

53. Timeout

Every parallel branch needs a timeout.

```
{
  "flight_search_timeout_seconds": 15,
  "hotel_search_timeout_seconds": 15,
  "policy_search_timeout_seconds": 8
}
```

Without timeouts, one branch may block the entire fan-in indefinitely. Timeout outcomes should be explicit:

```
{
  "branch": "hotel_search",
  "status": "timed_out"
}
```

54. Global and Branch Timeouts

Branch timeout: Maximum time for one branch.

Global timeout: Maximum time for the entire parallel stage or run.

```
Each source: Maximum 10 seconds
Entire research stage: Maximum 20 seconds
```

A branch may still have time remaining when the global deadline is reached. The orchestrator should cancel or safely ignore unfinished branches.

55. Cancellation Propagation

When the parent run is cancelled, active child tasks should also stop where possible.

```
User cancels trip search
↓ Parent run cancelled
↓ Cancel flight search
  Cancel hotel search
  Cancel policy search
```

For write operations already submitted, cancellation may not reverse them. The workflow must check operation status and potentially run compensation.

56. Retry in Parallel Workflows

Each branch may have its own retry policy.

```
{
  "flight_search": { "max_retries": 2 },
  "legal_specialist": { "max_retries": 1 },
  "payment_write": {
    "blind_retry_allowed": false
  }
}
```

Retries should not cause: Duplicate writes, Excessive total cost, Repeated no-progress loops, or Barrier starvation.

57. Retry Storm

A retry storm occurs when many failed branches retry simultaneously.

```
100 parallel searches
↓ Service unavailable
↓ All 100 retry immediately
→ Worsens the outage
```

Defences include:

- Exponential backoff
- Randomized jitter
- Global concurrency limits
- Circuit breakers
- Shared service-health state
- Retry budgets

58. Concurrency Limit

Even independent work should not expand without limit.

```
# Unsafe: Launch 1,000 model calls simultaneously
# Safer: process with concurrency = 10
# → bounded concurrency
```

This controls:

- Cost
- Rate limits
- Memory use
- Service load
- Failure amplification

59. Work Queue

Large parallel workloads can use a queue.

```
Pending items
↓ Worker pool
↓ Completed results
```

```
Example: 500 invoices
↓ 20 workers extract data
↓ Results stored independently
```

A queue allows: Retries, Backpressure, Prioritization, Worker recovery, Progress tracking.

60. Backpressure

Backpressure slows or limits producers when downstream systems cannot keep up.

```
Agent generates 500 subtasks
↓ Worker capacity is 20
```

The orchestrator should queue or reject excess work rather than launching everything.

Backpressure protects: External APIs, Databases, Model limits, Memory, Budget.

61. Rate Limiting

Parallel branches must still respect shared limits.

```
{
  "maximum_concurrent_searches": 5,
  "maximum_searches_per_minute": 20,
  "maximum_total_searches_per_run": 12
}
```

A branch-level limit alone is insufficient if many branches call the same service.

62. Shared State

Parallel branches may need access to shared run state.

Risky design: Every branch reads and rewrites the complete state object. This can cause lost updates.

Safer design:

- Give each branch a scoped input snapshot.
- Let branches return structured outputs.
- Have the orchestrator merge results.
- Restrict branch-owned fields.

63. Branch-Local State

Each parallel branch should normally maintain local state.

```
{
  "branch_id": "legal_analysis",
  "status": "running",
  "tool_call_count": 2,
  "findings": [],
  "last_error": null
}
```

The parent run maintains per-branch status:

```
{
  "branches": {
    "legal_analysis": {"status": "running"},
    "financial_analysis": {"status": "completed"},
    "technical_analysis": {"status": "running"}
  }
}
```

64. State Merge

When a branch finishes, its output is merged into parent state.

```
{
  "branch_results": {
    "legal": {
      "status": "completed",
      "risk_level": "high"
    }
  }
}
```

The merge should be controlled by code. Do not allow a child agent to overwrite: Parent authorization, Overall run status, Other branch results, Global budget, or Approval state.

65. Lost Update Problem

Initial state: legal_result: null, financial_result: null

Legal branch saves: legal_result: "high risk", financial_result: null

Financial branch (old snapshot) saves: legal_result: null, financial_result: "positive" — **the legal result is lost.**

Solutions:

- Field-level updates
- Optimistic concurrency
- Event sourcing
- Branch-specific storage
- Reducer functions

66. Reducers

A reducer defines how several updates to one field are combined.

```
def merge_branch_results(current, update):
    return {**current,
            update["branch_id"]: update["result"]}
```

```
def append_messages(current, new):
    return current + new
```

```
def retain_highest_risk(current, new):
    return max(current, new, key=risk_rank)
```

Reducers should preserve invariants.

67. Immutable Inputs

A branch should often receive an immutable input snapshot.

```
{
  "goal": "analyse proposed acquisition",
  "company_id": "COMP-17",
  "data_snapshot_version": 42
}
```

The branch cannot silently modify the shared starting facts. If the environment changes during execution, the parent decides whether revalidation is required.

68. Snapshot Consistency

Parallel analysis should ideally use compatible data snapshots.

Financial branch: Uses Q2 data version 7.

Market branch: Uses Q3 data version 9.

Combining these without disclosure may create misleading conclusions. The parent may require:

```
{ "required_data_version": 9 }
```

or record different versions explicitly.

69. Parallel Tool Calls from One Model Turn

A model may propose several independent tool calls at once.

- get_weather_for_city_A
- get_weather_for_city_B
- get_weather_for_city_C

The orchestrator should verify that: Calls are independent, Every call is permitted, Shared limits are respected, Results are matched to call IDs, Errors are handled separately.

Do not assume that multiple tool calls are safe merely because the model proposed them together.

70. Sequential Tool Calls

Some tool calls must remain ordered.

```
search_customer_orders
↓ get_selected_order
↓ create_return
```

The return tool needs a verified order obtained earlier. Calling all three at once would require guessing the order.

71. Speculative Parallelism

Speculative parallelism starts work that may later prove unnecessary.

```
Search replacement inventory
while:
    Check whether replacement is policy-eligible
```

Speculation may be acceptable when:

- The calls are read-only.
- Cost is low.
- Latency reduction matters.
- No sensitive data is unnecessarily retrieved.
- The branch can be cancelled.

72. Risks of Speculative Execution

Speculative execution can cause:

- Wasted cost
- Unnecessary data access
- Rate-limit consumption
- Increased prompt-injection exposure
- Confusing traces
- Accidental side effects

Never speculatively execute consequential writes.

Unsafe: Begin refund while waiting for eligibility check.

73. Pipelining

Pipelining overlaps stages across different items.

```
Doc 1: Extract → Summarize → Validate
Doc 2: Extract → Summarize → Validate
Doc 3: Extract → Summarize → Validate
```

Different stages operate concurrently on different items. Pipelining can improve throughput without making dependent stages for the same item parallel.

74. Batch Processing

A workflow may process several items in one tool call.

```
# Instead of:
get_order(01)
get_order(02)
get_order(03)
```

```
# Use:
get_orders([01, 02, 03])
```

Batching may reduce: Network overhead, Tool-call count, Latency, Model context turns. But batch size should be bounded.

75. Batch versus Parallel Calls

Batching: One call processes several inputs.

Parallelism: Several calls process inputs concurrently.

Batching is often simpler and cheaper. Parallelism may be preferable when:

- Different tools are required.
- Individual timeouts matter.
- Items are large.
- Failures should remain isolated.
- The service does not support batching.

76. Sequential Approval Pattern

Consequential actions should follow:

```
Prepare action
↓ Present exact details
↓ Pause
↓ Receive approval
↓ Revalidate
↓ Execute
↓ Verify
```

Approval and execution should not be parallel. Confirmation must precede the exact action it authorizes.

77. Parallel Approval Requests

A workflow may request several independent approvals in parallel.

```
Legal approval ██████
Security approval ██████ All required
Finance approval █████
```

The write executes only after the required approval policy is satisfied. Possible policies: All approvals required, Any one authorized approver, Two of three, Role-specific combination. These rules should be deterministic.

78. Human Response Timing

Human approvals may arrive in any order. State should store each separately.

```
{
  "approvals": {
    "legal": "approved",
    "security": "pending",
    "finance": "approved"
  }
}
```

The workflow should not interpret "Two approvals arrived" as sufficient unless the policy explicitly permits it.

79. Sequential Verification

After parallel writes, verification may need to be coordinated.

```
Flight booking ████
Hotel booking ██████ Verify each result
Car booking ██████
↓ Validate complete itinerary
```

The overall workflow is complete only when: Every required booking has a valid confirmation, Dates and locations are consistent, No duplicate charge exists, The final itinerary is coherent.

80. Result Correlation

Parallel tool results must be matched to their originating requests.

```
{
  "call_id": "CALL-882",
  "branch_id": "flight_search",
  "request_id": "REQ-17"
}
```

Without correlation IDs, results may be attached to the wrong branch or item.

81. Out-of-Order Results

Searches launched as A, B, C may return as C, A, B. The system must merge by ID:

```
results[result.branch_id] = result # not by arrival
```

82. Duplicate Result Delivery

Distributed systems may deliver a branch result more than once. Use:

- Unique event IDs
- Idempotent state updates
- Deduplication
- Sequence numbers

```
{
  "event_id": "EVENT-771",
  "branch_id": "legal",
  "result_version": 1
}
```

The parent should not append the same finding twice.

83. Branch Completion Statuses

Useful statuses include:

```
PENDING  RUNNING  COMPLETED
FAILED    TIMED_OUT CANCELLED
SKIPPED   PARTIALLY_COMPLETED
```

```
{
  "branches": {
    "flight_search": "completed",
    "hotel_search": "timed_out",
    "policy_search": "completed"
  }
}
```

84. Parent Completion Policy

The parent needs explicit completion rules. Example:

Complete only when: Policy branch completed. At least one flight option exists. At least one hotel option exists. Every presented option passes policy checks.

The parent should not finish merely because every branch stopped. A failed branch is not equivalent to a successful branch.

85. Parallel Research Example

Goal: Assess whether a company should enter a new market.

Fan-out:

- Market-size analysis
- Competitor analysis
- Regulatory analysis
- Customer-demand analysis
- Operational feasibility

Fan-in:

- Validate evidence
 - ↓ Identify agreements and conflicts
 - ↓ Calculate deterministic metrics
 - ↓ Synthesize recommendation

Completion: Every mandatory domain analysed, Evidence references present, Major conflicts disclosed, Recommendation tied to findings, Uncertainty stated.

86. Customer-Support Example

Goal: Resolve a missing delivery.

Sequential start:

Authenticate customer → Retrieve order

Parallel investigation:

- Retrieve carrier tracking
- Retrieve delivery photo
- Read missing-delivery policy

Sequential decision:

Combine evidence
 → Determine eligible resolutions
 → Ask customer → Confirm → Execute → Verify

This is a hybrid workflow.

87. Software-Engineering Example

Goal: Fix a failing application test.

Sequential:

Read failing test → Inspect error → Identify f.

Parallel:

- Search related code
- Inspect recent changes
- Review dependency behaviour

Sequential:

Propose patch → Apply → Run targeted test
 → Run regression tests → Complete or revise

Do not apply several conflicting patches to the same branch in parallel without isolation.

88. Isolated Parallel Coding Branches

Several coding agents may test different fixes in separate sandboxes or branches.

Base repository

- Branch A: Fix strategy 1
- Branch B: Fix strategy 2
- Branch C: Fix strategy 3

Each runs tests independently. An evaluator compares: Correctness, Regression results, Complexity, Changed files, Security implications. Only one approved change is merged.

89. Parallel Candidate Generation

A model or several agents may generate multiple candidate plans (Candidate A, B, C), then an evaluator ranks them.

This can improve quality for difficult tasks. Risks include:

- Increased cost
- Correlated errors
- Evaluator bias
- Excessive complexity

A candidate is not an authorized action.

90. Sequential Refinement

Instead of parallel candidates, one agent may revise sequentially.

Draft → Evaluate → Feedback → Revise → Re-evaluate

Sequential refinement is useful when each revision depends on previous feedback. Use a maximum revision count to prevent infinite loops.

91. Parallel Evaluation

Several evaluators may assess one output independently.

Output

- Safety evaluator
- Factuality evaluator
- Policy evaluator
- Style evaluator

These evaluations can run in parallel because each receives the same output. The final decision may require: Every hard safety check to pass, Deterministic policy checks to pass, Soft-quality scores above thresholds.

92. Hard Gates and Soft Scores

A parallel evaluation stage may produce:

```
{
  "safety_pass": true,
  "policy_pass": true,
  "citation_score": 0.92,
  "clarity_score": 0.84
}
```

Hard gate: If safety_pass is false → reject.

Soft criterion: If clarity_score is low → revise.

Do not average a severe safety failure away with high style scores.

93. Parallelism and Tool Permissions

Every branch should receive only the capabilities it requires.

Example: Legal branch → legal document search.
Financial branch → financial data retrieval, calculator.
Technical branch → architecture documents, code search.

None should automatically receive: Refund tool, Email tool, Production deployment, Customer-record deletion.

94. Parallelism and Data Isolation

Branches should receive only relevant data.

Technical specialist needs: System architecture and performance data. It does not need: Employee salary records.

Parallelism multiplies the number of components handling data, so poor isolation increases privacy exposure.

95. Parallelism and Prompt Injection

Each retrieval branch may encounter malicious content. More branches mean a larger attack surface.

Defences include:

- Source labelling
- Restricted tools
- No external-send capability
- Sanitized results
- Aggregator instructions that treat branch content as evidence
- Policy validation before action

A malicious document found by one branch must not become a parent instruction.

96. Workflow Observability

Record:

- Workflow ID, Node/Step ID, Branch ID
- Dependencies, Start and completion times
- Inputs and outputs, Retry count, Error status
- Parent–child relationships, State versions
- Aggregation decision, Stop reason

```
{
  "workflow_id": "WF-482",
  "node_id": "legal_analysis",
  "branch_id": "BR-LEGAL",
  "status": "completed",
  "duration_ms": 4200
}
```

97. Critical Path

The critical path is the longest chain of dependent work that determines the minimum workflow duration.

A: 2s B: 5s after A C: 3s after A D: 4s after A

$A \rightarrow B \rightarrow D = 11s$ (critical path)

$A \rightarrow C \rightarrow D = 9s$

Making C faster will not reduce total latency unless it becomes slower than B.

98. Optimizing the Critical Path

To reduce total latency:

- Parallelize genuinely independent tasks.
- Speed up steps on the critical path.
- Cache safe read results.
- Remove unnecessary dependencies.
- Use faster models for simple stages.
- Batch repeated calls.
- Begin low-risk speculative reads carefully.

Do not remove required verification merely to reduce latency.

99. Workflow Cost

Parallel execution may reduce time but increase cost.

Sequential: Run one search. Use its result to decide whether another is needed.

Parallel: Run five searches immediately. Parallel version may finish faster but perform unnecessary work.

The design balances: Latency, Cost, Coverage, Risk.

100. Cost-Aware Fan-Out

A parent may limit branch creation.

```
{
  "maximum_parallel_branches": 4,
  "maximum_total_child_cost": 2.00,
  "maximum_model_calls_per_branch": 3
}
```

The model may propose ten subquestions, but the orchestrator selects only the permitted number.

101. Adaptive Parallelism

The workflow may change concurrency based on conditions.

System healthy: concurrency = 10

Service degraded: concurrency = 3

Rate limit approaching: concurrency = 1

Adaptive parallelism is controlled by the orchestrator, not improvised by individual agents.

102. Dynamic Fan-Out

An agent may generate subquestions dynamically.

```
{"subtasks": [
  "Analyse traffic",
  "Analyse conversion",
  "Analyse pricing",
  "Analyse churn",
  "Analyse payment failures"
]}
```

Before fan-out, the application should validate: Subtasks are relevant, No duplicates exist, Count is within budget, Required data permissions exist, Each goal is bounded.

103. Static versus Dynamic Workflow

Static workflow: Steps and dependencies are defined in advance ($A \rightarrow B \rightarrow C$). Advantages: Predictable, Easy to test, Strong governance.

Dynamic workflow: The agent proposes tasks or dependencies at runtime. Advantages: Flexible, Handles novel goals. Risks: Unbounded branching, Duplicate work, Unsafe tasks, Difficult evaluation.

Production systems often use: Dynamic choices inside a static outer workflow.

104. Static Outer, Dynamic Inner

Static outer workflow:

- 1. Interpret question
- 2. Generate bounded research plan
- 3. Execute approved research branches
- 4. Aggregate
- 5. Validate citations
- 6. Finish

Dynamic inner: Which subquestions to investigate, Which sources to search, How to synthesize evidence.

The outer workflow enforces: Budget, Permissions, Maximum branches, Completion, Safety.

105. Sequential and Parallel Composition

Workflows can be nested.

```
Stage 1: Sequential authentication
  ↓ Stage 2: Parallel information collection
  ↓ Stage 3: Sequential decision and approval
  ↓ Stage 4: Parallel independent fulfilment
  ↓ Stage 5: Sequential verification
```

This is common in real applications.

106. Complete Business-Travel Workflow

Goal: Arrange an approved business trip.

```
Stage 1 (Sequential requirements):
  Collect destination → dates → business purpose
```

```
Stage 2 (Parallel retrieval):
  ■■■ Search flights
  ■■■ Search hotels
  ■■■ Retrieve travel policy
```

```
Stage 3 (Sequential filtering):
  Validate → Apply policy → Rank itineraries
```

```
Stage 4: Present → User selects → Manager approval
Stage 5: Refresh prices and availability
```

```
Stage 6 (Parallel booking):
  Book flight ■■■
  Book hotel ■■■■■■ Monitor results
```

```
Stage 7: Both succeed?
  Yes → Verify and send itinerary
  No  → Retry, compensate or escalate
```

107. Complete Damaged-Order Workflow

Goal: Replace a damaged monitor.

```
Sequential:
  Authenticate customer → Retrieve order
```

```
Parallel reads:
  ■■■ Verify delivery details
  ■■■ Check replacement policy
  ■■■ Check current inventory
```

```
Sequential:
  Collect damage evidence
  → Determine eligible resolution
  → Present replacement option
  → Receive confirmation
```

```
Write: Create replacement
```

```
Sequential verification:
  Retrieve replacement order
  → Verify address and product → Complete
```

108. Workflow Definition Example

```
from dataclasses import dataclass, field
from typing import Any

@dataclass
class NodeResult:
    status: str
    data: dict[str, Any] = field(default_factory=dict)
    error: dict[str, Any] | None = None

async def plan_business_trip(requirements):
    validated = validate_requirements(requirements)
    flight_task = search_flights(validated)
    hotel_task = search_hotels(validated)
    policy_task = retrieve_travel_policy(validated)

    flight_result, hotel_result, policy_result = \
        await gather_safely(
            flight_task, hotel_task, policy_task)

    ensure_required_branch_succeeded(
        branch="travel_policy",
        result=policy_result,
    )
    permitted_options = apply_policy(
        flights=flight_result.data,
        hotels=hotel_result.data,
        policy=policy_result.data,
    )
    return {
        "status": "awaiting_user_selection",
        "options": rank_itineraries(permitted_options),
    }
```

The parallel retrieval stage is surrounded by deterministic validation.

109. Safe Gather Function

```
async def gather_safely(*tasks):
    results = []
    for completed_task in run_with_bounded_concurrency(
        tasks, maximum_concurrency=3):
        try:
            result = await completed_task
        except TimeoutError:
            result = NodeResult(
                status="timed_out",
                error={"code": "BRANCH_TIMEOUT",
                    "retryable": True})
        except Exception:
            result = NodeResult(
                status="failed",
                error={"code": "UNEXPECTED_BRANCH",
                    "retryable": False})
        results.append(result)
    return results
```

A production implementation should preserve branch identity and cancellation semantics.

110. Workflow State Example

```
{
    "workflow_id": "WF-482",
    "status": "parallel_retrieval",
    "current_stage": 2,
    "branches": {
        "flight_search": {"status": "completed", "attempt": 1},
        "hotel_search": {"status": "running", "attempt": 1},
        "policy_retrieval": {"status": "completed", "attempt": 1}
    },
    "required_branches": [
        "flight_search", "hotel_search", "policy_retrieval"
    ],
    "maximum_concurrency": 3,
    "state_version": 8
}
```

111. Workflow Completion Validation

```
def trip_workflow_complete(state) -> bool:
    return (
        state.flight_booking_verified
        and state.hotel_booking_verified
        and state.manager_approval_valid
        and state.total_cost_within_approved_limit
        and state.itinerary_sent
    )
```

All branches reaching a terminal state is not enough. The required business outcome must be verified.

112. Common Sequential Workflow Mistakes

- Mistake 1:** Unnecessary serialization — independent searches run one after another.
- Mistake 2:** Missing prerequisite — a write runs before identity verification.
- Mistake 3:** Overly rigid flow — the workflow cannot handle a valid exception.
- Mistake 4:** Passing huge outputs between every stage — context and storage become inefficient.
- Mistake 5:** No checkpointing — a late failure forces complete restart.
- Mistake 6:** No recovery path — any error terminates the whole process.

113. Common Parallel Workflow Mistakes

- Mistake 1:** Parallelizing dependent tasks — a later task uses guessed inputs.
- Mistake 2:** Parallel writes to the same resource — state conflicts occur.
- Mistake 3:** Unbounded fan-out — cost and service load explode.
- Mistake 4:** No synchronization policy — the aggregator does not know when to continue.
- Mistake 5:** Ignoring partial failures — missing evidence is silently omitted.
- Mistake 6:** Shared-state overwrites — branch results are lost.
- Mistake 7:** Treating completion order as meaningful — results attached to the wrong task.
- Mistake 8:** No cancellation propagation — child tasks continue after user cancels.
- Mistake 9:** No idempotency — retries duplicate side effects.
- Mistake 10:** Model-only aggregation — invalid or malicious branch output accepted without validation.

114. Design Questions

- Before making steps sequential, ask:**
- Does one step require another's data?
 - Does one step determine whether another is permitted?
 - Is a strict business order required?
 - Will parallel execution create stale assumptions?
 - Do the steps modify the same resource?
- Before making steps parallel, ask:**
- Are the inputs independently available?
 - Are the branches read-only or isolated?
 - Can branch failures be handled independently?
 - How will results be synchronized?
 - What is the concurrency limit?
 - How will shared state be merged?
 - How will cancellation and retries work?

115. Sequential versus Parallel Decision Table

Retrieve order → calculate refund	Sequential
Search three independent databases	Parallel
Obtain approval → issue refund	Sequential
Analyse legal/financial/technical risks	Parallel
Modify same customer record twice	Sequential
Summarize 100 independent documents	Parallel
Create appointment → verify it	Sequential
Run safety and style evaluations	Parallel
Search flights and hotels	Parallel
Determine target order → cancel it	Sequential

116. Production Mental Model

Sequential workflow: Order creates correctness.

Parallel workflow: Independence creates speed.

Fan-out: Divide work into bounded branches.

Barrier: Wait for the required branch conditions.

Fan-in: Validate and combine results.

Compensation: Respond safely when a distributed write partially succeeds.

Bounded concurrency: Use parallelism without losing control.

Parallelism is not simply "do everything at once." It is a controlled optimization applied only after dependencies, state ownership, failure behaviour, budgets and synchronization rules are clearly defined.

Checkpoint 16

Part A: Definitions

Explain each term in one or two sentences:

- Sequential workflow
- Parallel workflow
- Data dependency
- Control dependency
- Fan-out
- Fan-in
- Barrier
- Bounded concurrency
- Compensation
- Critical path

Part B: Sequential or parallel?

Classify each pair as: Sequential / Parallel / Depends on implementation

- Retrieve an order and calculate its refundable amount.
- Search flight and hotel options.
- Obtain approval and execute a refund.
- Analyse legal and technical risks.
- Update an address and cancel the same order.
- Summarize ten independent documents.
- Book a flight and hotel.

- Run safety and style evaluations on the same answer.
- Retrieve current appointment details and retrieve branch opening hours.
- Create an appointment and verify the appointment.

Part C: Identify the dependency

For each scenario, identify whether the main dependency is: Data dependency / Control dependency / Authorization dependency / State dependency / No dependency

- A refund amount is calculated after retrieving the amount paid.
- A refund executes only if the order is eligible.
- Private payroll data is retrieved only after authentication.
- Booking is permitted only from READY_TO_EXECUTE.
- Flight search and hotel search use the same trip requirements.
- An email is sent only after the draft receives approval.

Part D: Build a DAG

Create a workflow graph for: Prepare and publish a quarterly business report. Include:

- Retrieve financial data
- Retrieve sales data
- Retrieve customer data
- Validate datasets
- Analyse financial performance
- Analyse sales performance
- Analyse customer trends
- Generate report draft
- Run factuality review
- Run policy review
- Obtain executive approval
- Publish report

Identify: Which steps are parallel, Which are sequential, Synchronization barriers, The likely critical path.

Part E: Partial-failure policy

A travel workflow runs these branches: flight_search, hotel_search, car_search, travel_policy.

Results: flight_search = success, hotel_search = success, car_search = success, travel_policy = success.

Design the recovery policy. Explain:

- Whether the workflow should fail immediately
- Whether hotel search should retry
- Whether successful results should be preserved
- Whether the user may receive partial options
- When human escalation is required
- Which branch is mandatory before booking

Part F: Parallel-state exercise

Three branches update shared state: Legal branch adds legal risks. Financial branch adds cost projections. Technical branch adds architecture constraints.

Design a state structure and merge policy that prevents one branch from overwriting the others. Include:

- Branch IDs
- Branch-local state
- Parent state
- State versions
- Reducer or merge function
- Duplicate-result protection

Part G: Saga exercise

Design a travel-booking saga with: Flight booking, Hotel booking, Car booking, Payment, Final itinerary confirmation.

For each step, define:

- Forward action
- Success evidence
- Compensation action
- Idempotency strategy
- Behaviour when compensation fails

Part H: Map-reduce exercise

Design a workflow that analyses 100 customer-support tickets. Define:

- Map input
- Parallel worker task
- Worker output schema
- Maximum concurrency
- Retry policy

- Reduce operation
- How duplicate findings are removed
- How evidence is preserved
- How failed tickets are represented
- Completion criteria

Part I: Find the unsafe parallelism

Consider:

- 1. Agent identifies a likely order.
- 2. In parallel: It verifies order ownership, It issues a refund, It retrieves the refund policy.
- 3. It combines the results.

Explain every design error and provide the correct ordering.

Part J: Reasoning question

Explain why parallel execution can improve latency while simultaneously reducing reliability if dependencies, shared state, cancellation, idempotency and partial failures are not carefully controlled.

The next lesson is Conditional Branching and Decision Workflows, where we will study if/else paths, rule tables, model-selected branches, loop boundaries, and safe exception handling.